LAPORAN PRAKTIKUM SPARK DAN RDD



Dosen Pengampu:
Ersha Aisyah Elfaiz, S.Si., M.T.I.

Disusun oleh:
Muhammad Anwar Ismail (24051204161)

Program Studi Teknik Informatika
Fakultas Teknik
Universitas Negeri Surabaya
2026

DAFTAR ISI

DAFTAR ISI ..... **i**
DAFTAR GAMBAR **1**
BAB I PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Tujuan Praktikum 3
BAB II DASAR TEORI **4**
2.1 Konsep Resilient Distributed Datasets (RDD) dan Karakteristiknya 4
2.2 Komparasi HDFS Block (Lapisan Storage) vs RDD Partition (Lapisan Processing) 4
2.3 Klasifikasi Operasi RDD (Transformations dan Action) 4
2.4 Mekanisme Lazy Evaluation dan Persistensi Memori (Caching) 4
2.5 Arsitektur Klaster Apache Spark Standalone 4
2.6 Lingkungan Interaktif Spark Shell dan Ekosistem Scala 4
2.7 Pengawasan Klaster dan Aplikasi melalui Spark Web UI 4
2.8 Komparasi Model Deployment (Spark Standalone vs Spark on YARN) 4
BAB III PEMBAHASAN **6**
3.1 Hasil dan Tangkapan Layar RDD, Transformation, Action, dan Partition pada Apache Spark 6
3.2 Hasil dan Tangkapan Layar Instalasi dan Menjalankan Apache Spark Standalone di Windows 16
3.3 Analisis Mekanisme Partisi dan Distribusi Data (glom) 21
3.4 Perbandingan Kinerja dan Karakteristik (map() vs flatMap()) 21
3.5 Perbedaan Analitis (reduce() vs reduceByKey()) 21
3.6 Pembuktian Empiris Lazy Evaluation dan Efektivitas Caching 22
3.7 Hubungan HDFS Storage Layer dengan Spark Processing Layer 22
BAB IV KESIMPULAN **24**
4.1 Kesimpulan 24
LAMPIRAN A COMMAND UTAMA **25**
LAMPIRAN B PORT YANG DIGUNAKAN **27**

DAFTAR GAMBAR

Gambar 3.1.1 - Status Docker Desktop dan versi Docker Engine serta Docker Compose.png 6
Gambar 3.1.2 - Struktur direktori dan isi file konfigurasi docker-compose.spark.yml_1.png 7
Gambar 3.1.3 - Struktur direktori dan isi file konfigurasi docker-compose.spark.yml_2.png 7
Gambar 3.1.4 - Status container klaster Spark Standalone menggunakan docker ps.png 8
Gambar 3.1.5 - Antarmuka Spark Master Web UI pada localhost 8080.png..... 8
Gambar 3.1.6 - Antarmuka Spark Worker 1 Web UI pada localhost 8081.png..... 9
Gambar 3.1.7 - Antarmuka Spark Worker 2 Web UI pada localhost 8082.png..... 9
Gambar 3.1.8 - Pengecekan versi Apache Spark di dalam container spark-master.png 10
Gambar 3.1.9 - Inisialisasi dan koneksi spark-shell ke klaster Spark Standalone.png 10
Gambar 3.1.10 - Pembuatan RDD angka 1 sampai 20 dan pengecekan jumlah partisi default.png 11
Gambar 3.1.11 - Pembuatan RDD dengan 4 partisi eksplisit dan inspeksi partisi menggunakan glom.png 11
Gambar 3.1.12 - Eksekusi transformasi map untuk mengalikan nilai elemen dengan 5.png 11
Gambar 3.1.13 - Eksekusi transformasi filter untuk menyaring bilangan genap.png 11
Gambar 3.1.14 - Eksekusi transformasi flatMap untuk pemecahan baris kalimat menjadi kata.png..... 12
Gambar 3.1.15 - Eksekusi aksi count dan first pada RDD kata.png..... 12
Gambar 3.1.16 - Eksekusi aksi reduce untuk menghitung nilai akumulasi total elemen.png 12
Gambar 3.1.17 - Pemrosesan pipeline WordCount lokal menggunakan flatMap map dan reduceByKey.png 12
Gambar 3.1.18 - Verifikasi kesiapan file input.txt di dalam HDFS klaster Hadoop.png 13
Gambar 3.1.19 - Pembacaan dataset HDFS ke dalam RDD menggunakan sc.textFile.png 13
Gambar 3.1.20 - Eksekusi pipeline WordCount terdistribusi terhadap file HDFS.png 13
Gambar 3.1.21 - Pengujian mekanisme caching RDD menggunakan cache dan eksekusi dua aksi.png 14
Gambar 3.1.22 - Antarmuka Spark Application UI pada localhost 4040 tab Jobs.png 14
Gambar 3.1.23 - Antarmuka Spark Application UI tab Stages dan visualisasi DAG.png 15
Gambar 3.1.24 - Antarmuka Spark Application UI tab Executors.png..... 15

Gambar 3.1.25 - Antarmuka Spark Application UI tab Storage saat RDD di-cache.png 16
Gambar 3.1.26 - Proses penghentian sesi spark-shell dan shutdown klaster Docker.png 16
Gambar 3.2.1 - Status container klaster Spark Standalone 1 Master dan 2 Worker aktif.png 17
Gambar 3.2.2 - Antarmuka Spark Master Web UI pada localhost 8080 dengan dua Worker berstatus ALIVE.png 17
Gambar 3.2.3 - Menjalankan spark-shell terhubung ke Master Standalone.png 18
Gambar 3.2.4 - Pembuatan RDD data numerik dengan parallelize dan penayangan data via collect.png 18
Gambar 3.2.5 - Penentuan jumlah partition RDD dan inspeksi pembagian data via glom.png 18
Gambar 3.2.6 - Penerapan operasi transformasi map perkalian dua dan filter bilangan genap.png 19
Gambar 3.2.7 - Eksekusi program WordCount sederhana menggunakan reduceByKey.png 19
Gambar 3.2.8 - Pembacaan berkas input.txt dari HDFS menggunakan textFile.png 19
Gambar 3.2.9 - Eksekusi WordCount terhadap berkas dari HDFS.png 20
Gambar 3.2.10 - Antarmuka Spark Application UI pada port 4040.png 20
Gambar 3.2.11 - Penghentian sesi spark-shell.png 20

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pertumbuhan data yang sangat cepat menuntut kerangka kerja pemrosesan yang mampu bekerja secara paralel pada banyak mesin sekaligus. Apache Spark menjawab kebutuhan tersebut melalui abstraksi Resilient Distributed Datasets (RDD) yang memproses data di memori sehingga jauh lebih cepat dibandingkan model MapReduce berbasis disk. Praktikum ini melanjutkan pengenalan Spark Standalone berbasis Docker untuk melatih operasi RDD, pengamatan partisi, dan integrasi Spark dengan HDFS.

1.2 Tujuan Praktikum

1. Menjalankan klaster Spark Standalone dengan 1 Master dan 2 Worker menggunakan Docker serta memantaunya melalui Spark Web UI.
2. Menerapkan operasi RDD berupa parallelize, map, filter, flatMap, reduce, dan reduceByKey, serta mengamati pembagian partisi dengan getNumPartitions dan glom.
3. Membaca berkas dari HDFS menggunakan textFile, menjalankan WordCount, dan membuktikan konsep lazy evaluation serta caching.

BAB II
DASAR TEORI

2.1 Konsep Resilient Distributed Datasets (RDD) dan Karakteristiknya

RDD adalah abstraksi dataset terdistribusi pada Spark yang terbagi menjadi beberapa partisi dan diproses secara paralel oleh executor. RDD bersifat immutable dan dapat dipulihkan saat terjadi kegagalan karena Spark mencatat lineage transformasi yang membentuknya. Karakteristik ini menjadikan RDD fondasi pemrosesan data yang andal pada klaster.

2.2 Komparasi HDFS Block (Lapisan Storage) vs RDD Partition (Lapisan Processing)

HDFS Block membagi data demi kebutuhan penyimpanan dan tersimpan pada DataNode, sedangkan RDD Partition membagi data demi kebutuhan pemrosesan dan dikelola oleh Spark. Setiap partisi diproses sebagai satu task yang dijalankan oleh executor. Jumlah partisi tidak otomatis sama dengan jumlah DataNode maupun jumlah executor.

2.3 Klasifikasi Operasi RDD (Transformations dan Action)

Transformation seperti map, filter, flatMap, dan reduceByKey membentuk RDD baru dan bersifat lazy. Action seperti collect, count, first, dan reduce memicu eksekusi serta mengembalikan hasil ke driver. Pembagian ini menentukan kapan Spark benar-benar mulai bekerja.

2.4 Mekanisme Lazy Evaluation dan Persistensi Memori (Caching)

Spark menunda eksekusi seluruh transformation sampai sebuah action dipanggil, sehingga rangkaian operasi dapat dijalankan sebagai satu job yang efisien. Fungsi cache() menyimpan hasil RDD agar action berikutnya tidak perlu menghitung ulang lineage dari awal. Apabila memori tidak mencukupi, Spark dapat memakai mekanisme persistence lain sesuai resource yang tersedia.

2.5 Arsitektur Klaster Apache Spark Standalone

Spark Standalone terdiri dari Master yang mengatur resource, Worker yang menyediakan resource, Driver yang mengendalikan aplikasi, serta Executor yang menjalankan task. Driver meminta resource kepada Master, Master menugaskan Worker, kemudian Worker meluncurkan Executor. Praktikum ini membentuk klaster berisi 1 Master dan 2 Worker yang masing-masing berjalan dalam container Docker.

2.6 Lingkungan Interaktif Spark Shell dan Ekosistem Scala

Spark Shell adalah lingkungan interaktif berbasis Scala yang otomatis menyediakan SparkContext dalam variabel sc. Setiap baris kode langsung dieksekusi dan hasilnya langsung tampil, sehingga cocok untuk bereksperimen dengan operasi RDD. Sintaks Scala yang dipakai tergolong sederhana, yaitu val dan fungsi lambda.

2.7 Pengawasan Klaster dan Aplikasi melalui Spark Web UI

Spark Master UI pada port 8080 menampilkan Master URL, daftar Worker, serta penggunaan CPU dan memori klaster. Spark Worker UI pada port 8081 dan 8082 memperlihatkan kondisi masing-masing Worker. Spark Application UI pada port 4040 muncul selama aplikasi aktif dan menyajikan tab Jobs, Stages, Storage, dan Executors.

2.8 Komparasi Model Deployment (Spark Standalone vs Spark on YARN)

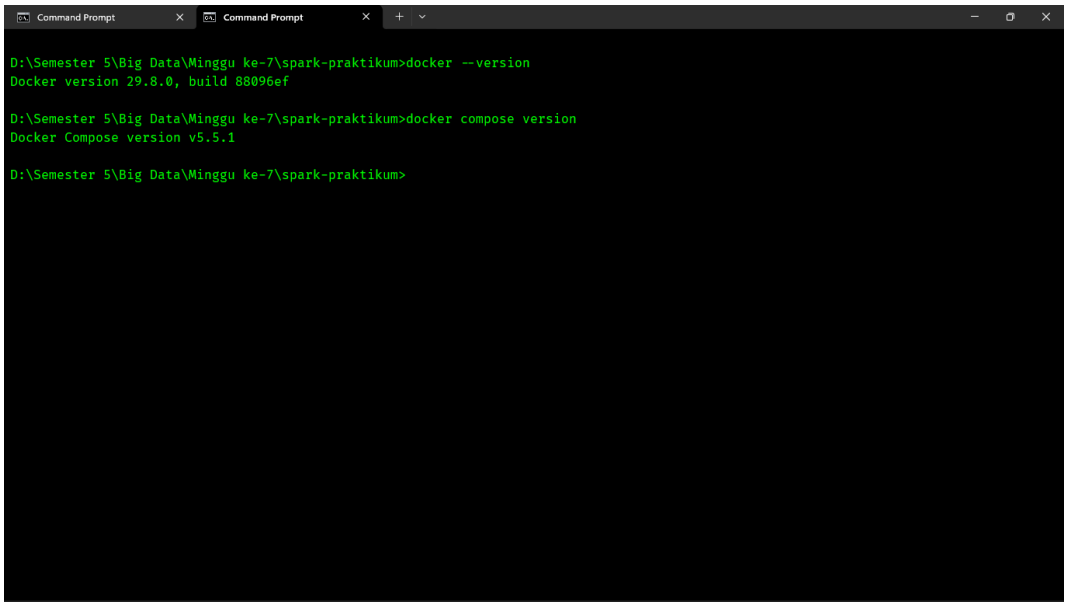
Spark Standalone memakai Spark Master sebagai cluster manager dengan alur Driver, Master, Worker, lalu Executor. Spark on YARN memakai ResourceManager dan

NodeManager dengan alur Driver, ResourceManager, Container, lalu Executor. Konsep RDD tetap sama pada kedua model, dan Standalone dipilih karena strukturnya lebih sederhana untuk dipelajari.

BAB III
PEMBAHASAN

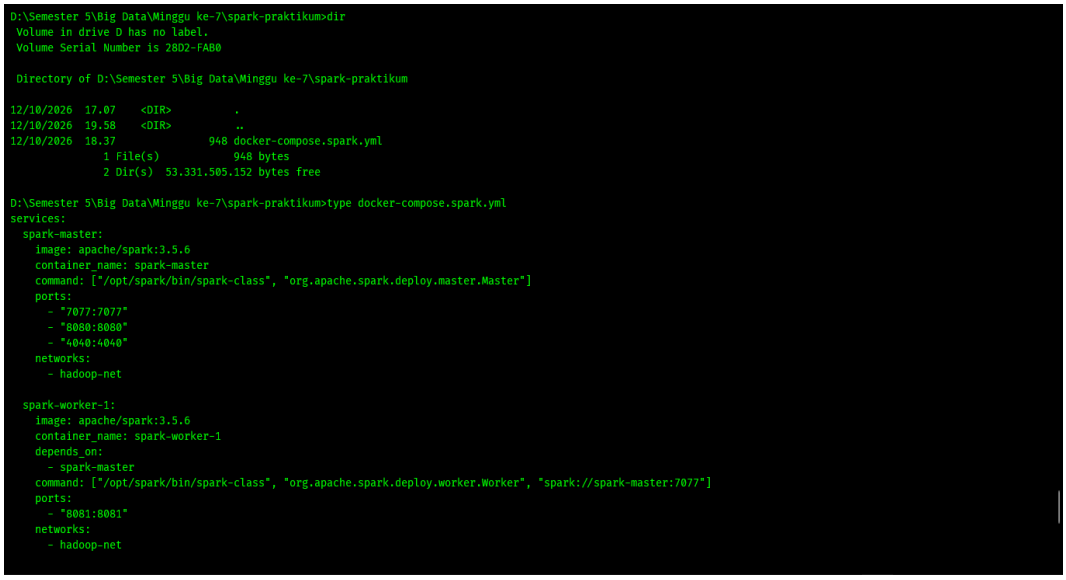
3.1 Hasil dan Tangkapan Layar RDD, Transformation, Action, dan Partition pada Apache Spark

Bagian ini menampilkan seluruh tahapan Praktikum 5, mulai dari pengecekan Docker, pembentukan klaster Spark, dan pembuatan RDD hingga eksekusi map, filter, flatMap, reduce, dan WordCount. Pembacaan berkas input.txt dari HDFS, pengujian cache, serta pengamatan Spark Application UI turut didokumentasikan pada rangkaian gambar. Setiap gambar dijelaskan secara berurutan pada uraian berikut.



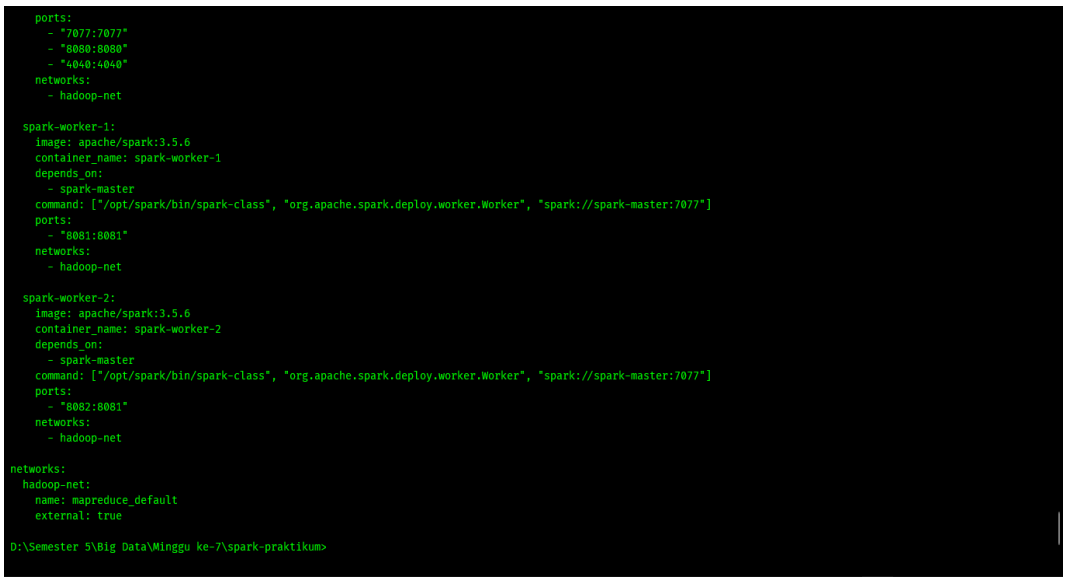
Gambar 3.1.1 - Status Docker Desktop dan versi Docker Engine serta Docker Compose.png

Perintah docker --version dan docker compose version menampilkan Docker 29.8.0 dan Docker Compose v5.5.1. Keluaran kedua versi tersebut menandakan Docker Engine sudah berjalan dan siap menjalankan container klaster.



Gambar 3.1.2 - Struktur direktori dan isi file konfigurasi docker-compose.spark.yml_1.png

Perintah dir memperlihatkan file docker-compose.spark.yml berukuran 948 bytes pada folder spark-praktikum. Perintah type kemudian menampilkan definisi service spark-master dengan image apache/spark:3.5.6, port 7077, 8080, dan 4040, serta jaringan hadoop-net.



Gambar 3.1.3 - Struktur direktori dan isi file konfigurasi docker-compose.spark.yml_2.png

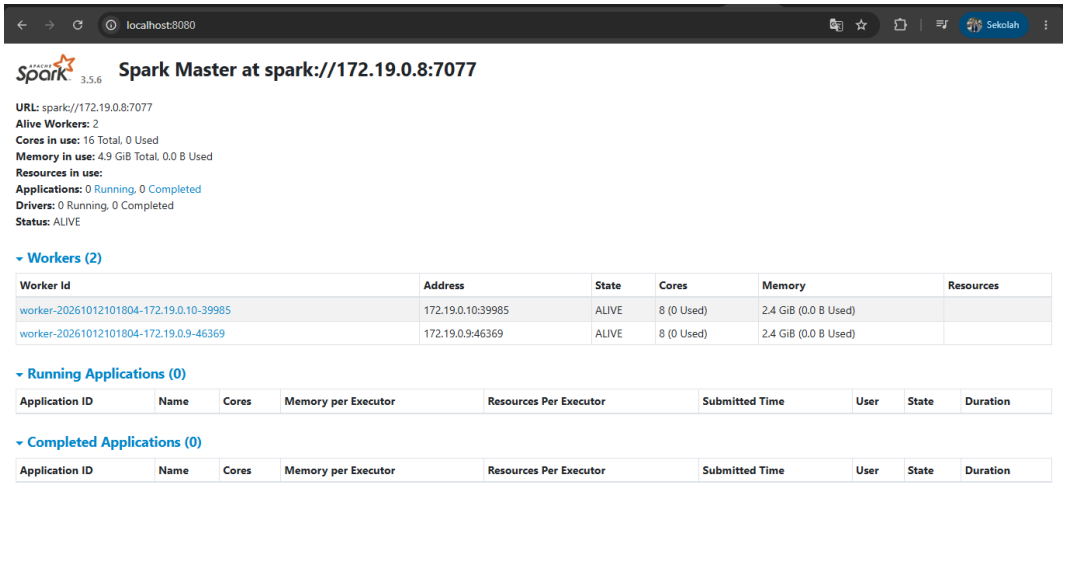
Bagian lanjutan file memperlihatkan service spark-worker-1 dan spark-worker-2 yang bergantung pada spark-master dan terhubung ke spark://spark-master:7077. Port Worker 1 dipetakan 8081:8081 dan Worker 2 dipetakan 8082:8081, sedangkan jaringan hadoop-net memakai jaringan eksternal mapreduce_default agar Spark dapat berkomunikasi dengan klaster Hadoop.

```
D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED       STATUS       PORTS
                                                                                          NAMES
93ce344ab3cc   apache/spark:3.5.6    "/opt/entrypoint.sh …"   2 hours ago   Up 2 hours   0.0.0.0:4040→4040/tcp, [::]:4040→4040/tcp,
 0.0.0.0:7077→7077/tcp, [::]:7077→7077/tcp, 0.0.0.0:8080→8080/tcp, [::]:8080→8080/tcp   spark-master
0826e86fd0a0   apache/spark:3.5.6    "/opt/entrypoint.sh …"   3 hours ago   Up 3 hours   0.0.0.0:8081→8081/tcp, [::]:8081→8081/tcp
                                                                                          spark-worker-1
242fa5136149   apache/spark:3.5.6    "/opt/entrypoint.sh …"   3 hours ago   Up 3 hours   0.0.0.0:8082→8081/tcp, [::]:8082→8081/tcp
                                                                                          spark-worker-2
18ecaf4fc698   apache/hadoop:3.4.3   "/usr/local/bin/dumb…"   2 weeks ago   Up 3 hours   0.0.0.0:8043→8042/tcp, [::]:8043→8042/tcp
                                                                                          nodemanager2
44c21fa4f471   apache/hadoop:3.4.3   "/usr/local/bin/dumb…"   2 weeks ago   Up 3 hours   0.0.0.0:8042→8042/tcp, [::]:8042→8042/tcp
                                                                                          nodemanager1
43b4e31ebd40   apache/hadoop:3.4.3   "/usr/local/bin/dumb…"   2 weeks ago   Up 3 hours
                                                                                          datanode2
71816ebb774d   apache/hadoop:3.4.3   "/usr/local/bin/dumb…"   2 weeks ago   Up 3 hours
                                                                                          datanode1
82258a1de82a   apache/hadoop:3.4.3   "/usr/local/bin/dumb…"   2 weeks ago   Up 3 hours   0.0.0.0:8088→8088/tcp, [::]:8088→8088/tcp
                                                                                          resourcemanager
aeb5602128a5   apache/hadoop:3.4.3   "/usr/local/bin/dumb…"   2 weeks ago   Up 3 hours   0.0.0.0:8020→8020/tcp, [::]:8020→8020/tcp,
 0.0.0.0:9870→9870/tcp, [::]:9870→9870/tcp                                                namenode

D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>
```

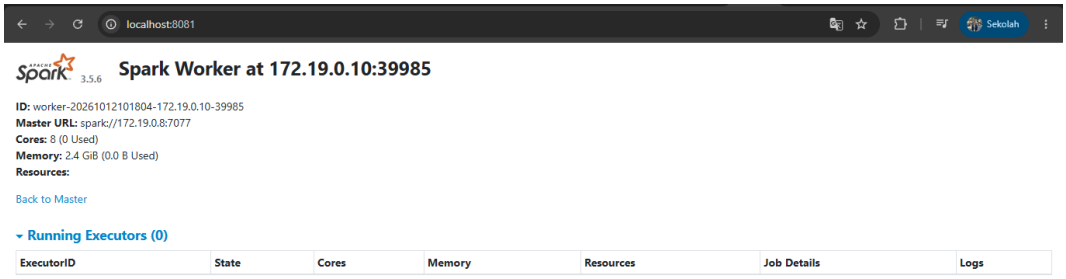
Gambar 3.1.4 - Status container klaster Spark Standalone menggunakan docker ps.png

Perintah docker ps menampilkan spark-master, spark-worker-1, dan spark-worker-2 berstatus Up. Container Hadoop dari praktikum sebelumnya, seperti namenode, datanode, resourcemanager, dan nodemanager, juga masih aktif sehingga HDFS tersedia untuk dibaca oleh Spark.



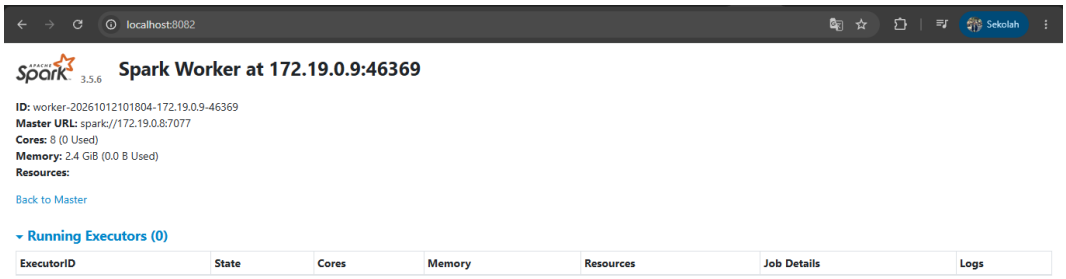
Gambar 3.1.5 - Antarmuka Spark Master Web UI pada localhost 8080.png

Spark Master UI menunjukkan Master URL spark://172.19.0.8:7077 dengan status ALIVE dan 2 Alive Workers. Total resource klaster tercatat 16 core dan 4,9 GiB memori, dengan belum ada aplikasi yang berjalan.



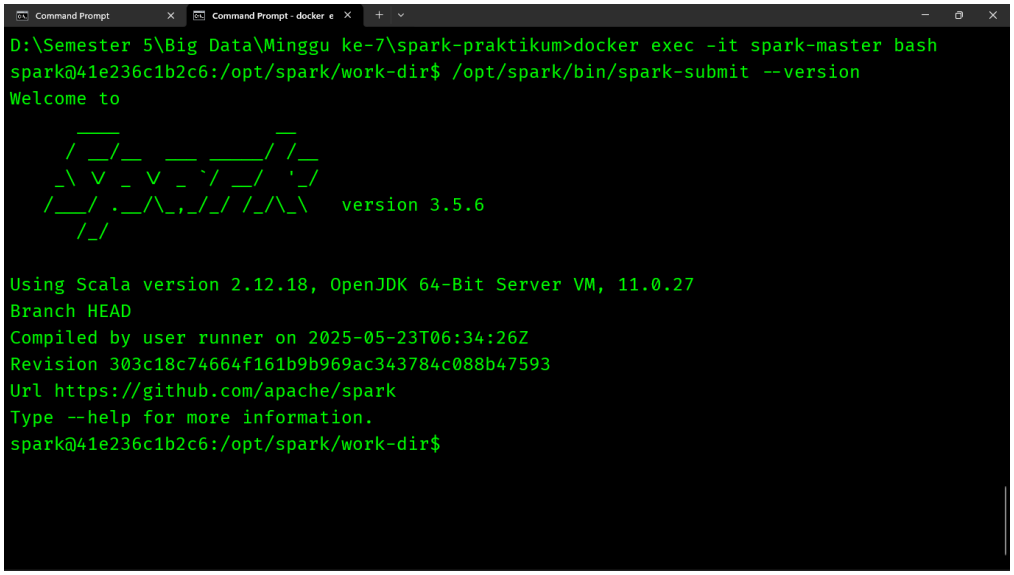
Gambar 3.1.6 - Antarmuka Spark Worker 1 Web UI pada localhost 8081.png

Worker 1 beralamat 172.19.0.10:39985 dengan resource 8 core dan memori 2,4 GiB. Bagian Running Executors masih kosong karena belum ada aplikasi Spark yang dijalankan.



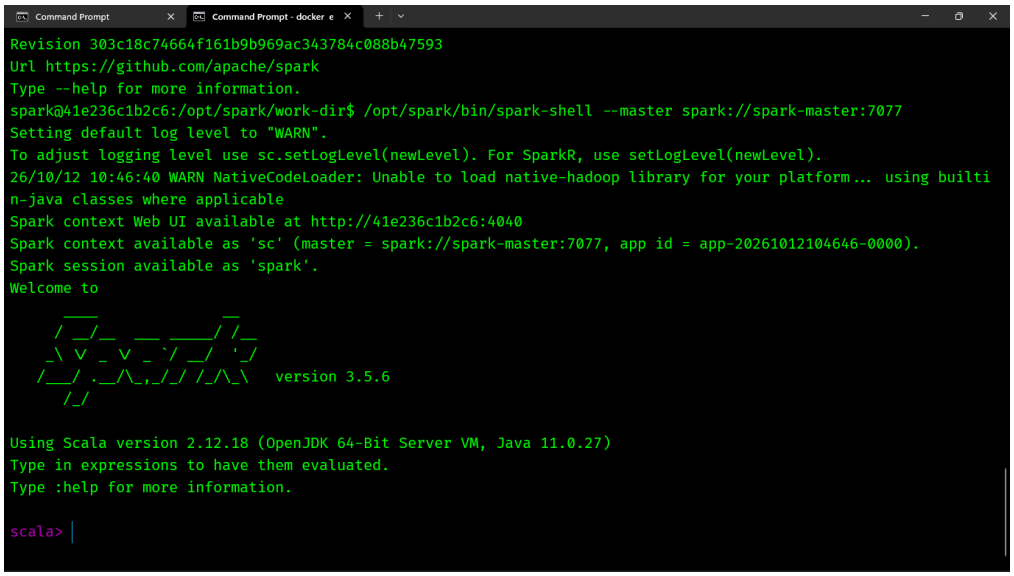
Gambar 3.1.7 - Antarmuka Spark Worker 2 Web UI pada localhost 8082.png

Worker 2 beralamat 172.19.0.9:46369 dengan resource yang sama, yaitu 8 core dan memori 2,4 GiB. Kedua Worker terhubung ke Master yang sama sehingga klaster memiliki kapasitas 16 core.



Gambar 3.1.8 - Pengecekan versi Apache Spark di dalam container spark-master.png

Perintah spark-submit --version menampilkan Apache Spark versi 3.5.6 yang berjalan dengan Scala 2.12.18 dan OpenJDK 11.0.27. Informasi ini mengonfirmasi versi Spark pada container sesuai dengan image yang dipakai.



Gambar 3.1.9 - Inisialisasi dan koneksi spark-shell ke klaster Spark Standalone.png

Spark Shell dijalankan dengan master spark://spark-master:7077 dan membentuk SparkContext sc dengan application ID app-20261012104646-0000. Spark Web UI aplikasi tersedia pada port 4040, dan prompt scala> menandakan shell siap menerima perintah.

```
scala> rdd.collect()
res0: Array[Int] = Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20)

scala> rdd.getNumPartitions
res1: Int = 16

scala>
```

Gambar 3.1.10 - Pembuatan RDD angka 1 sampai 20 dan pengecekan jumlah partisi default.png

Perintah collect() menampilkan seluruh isi RDD berupa angka 1 sampai 20. Perintah getNumPartitions menghasilkan nilai 16, yaitu jumlah partisi default yang sama dengan total core klaster.

```
scala> val rdd4 = sc.parallelize(1 to 20, 4)
rdd4: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[1] at parallelize at <console>:23

scala> rdd4.getNumPartitions
res2: Int = 4

scala> rdd4.glom().collect()
res3: Array[Array[Int]] = Array(Array(1, 2, 3, 4, 5), Array(6, 7, 8, 9, 10), Array(11, 12, 13, 14, 15), Array(1
6, 17, 18, 19, 20))

scala>
```

Gambar 3.1.11 - Pembuatan RDD dengan 4 partisi eksplisit dan inspeksi partisi menggunakan glom.png

RDD angka 1 sampai 20 dibuat dengan 4 partisi dan getNumPartitions menghasilkan nilai 4. Perintah glom().collect() memperlihatkan pembagian data menjadi [1-5], [6-10], [11-15], dan [16-20].

```
scala> val rddKali = rdd.map(x ⇒ x * 5)
rddKali: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[3] at map at <console>:23

scala> rddKali.collect()
res4: Array[Int] = Array(5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100)

scala>
```

Gambar 3.1.12 - Eksekusi transformasi map untuk mengalikan nilai elemen dengan 5.png

Transformation map(x => x * 5) membentuk RDD baru bertipe MapPartitionsRDD. Setelah collect() dipanggil, hasilnya berupa kelipatan lima dari 5 sampai 100 sebanyak 20 elemen.

```
scala> val rddGenap = rdd.filter(x ⇒ x % 2 == 0)
rddGenap: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[4] at filter at <console>:23

scala> rddGenap.collect()
res5: Array[Int] = Array(2, 4, 6, 8, 10, 12, 14, 16, 18, 20)

scala>
```

Gambar 3.1.13 - Eksekusi transformasi filter untuk menyaring bilangan genap.png

Transformation filter(x => x % 2 == 0) menyaring elemen yang memenuhi kondisi bilangan genap. Hasil collect() adalah 2, 4, 6, 8, 10, 12, 14, 16, 18, dan 20.

```
scala> val kalimat = sc.parallelize(Seq("Hadoop Spark", "Big Data Terdistribusi"))
kalimat: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[5] at parallelize at <console>:23

scala> val kata = kalimat.flatMap(line ⇒ line.split(" "))
kata: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[6] at flatMap at <console>:23

scala> kata.collect()
res6: Array[String] = Array(Hadoop, Spark, Big, Data, Terdistribusi)

scala>
```

Gambar 3.1.14 - Eksekusi transformasi flatMap untuk pemecahan baris kalimat menjadi kata.png

RDD berisi dua kalimat, yaitu "Hadoop Spark" dan "Big Data Terdistribusi", dipecah dengan flatMap dan split spasi. Hasilnya adalah lima kata yang berada pada satu RDD datar, yaitu Hadoop, Spark, Big, Data, dan Terdistribusi.

```
scala> kata.count()
res7: Long = 5

scala> kata.first()
res8: String = Hadoop

scala>
```

Gambar 3.1.15 - Eksekusi aksi count dan first pada RDD kata.png

Action count() menghasilkan nilai 5 yang sesuai dengan jumlah kata hasil flatMap. Action first() mengembalikan elemen pertama, yaitu kata Hadoop.

```
scala> val total = rdd.reduce((a, b) ⇒ a + b)
total: Int = 210

scala>
```

Gambar 3.1.16 - Eksekusi aksi reduce untuk menghitung nilai akumulasi total elemen.png

Action reduce((a, b) => a + b) menjumlahkan seluruh elemen RDD angka 1 sampai 20. Hasil yang muncul adalah satu nilai tunggal, yaitu 210.

```
scala> val teks = sc.parallelize(Seq("spark hadoop", "spark big data", "hadoop data"))
teks: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[7] at parallelize at <console>:23

scala> val words = teks.flatMap(line ⇒ line.split(" "))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[8] at flatMap at <console>:23

scala> val pairs = words.map(word ⇒ (word, 1))
pairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[9] at map at <console>:23

scala> val counts = pairs.reduceByKey((a, b) ⇒ a + b)
counts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[10] at reduceByKey at <console>:23

scala> counts.collect()
res9: Array[(String, Int)] = Array((big,1), (hadoop,2), (data,2), (spark,2))

scala>
```

Gambar 3.1.17 - Pemrosesan pipeline WordCount lokal menggunakan flatMap map dan reduceByKey.png

Teks berisi tiga kalimat dipecah menjadi kata, dipasangkan dengan angka 1, lalu dijumlahkan berdasarkan key melalui reduceByKey. Hasil akhirnya adalah (big,1),

(hadoop,2), (data,2), dan (spark,2), sedangkan RDD hasil reduceByKey bertipe ShuffledRDD.

```
D:\Semester 5\Big Data\Minggu ke-7>docker exec -it --user hadoop namenode /opt/hadoop/bin/hdfs dfs -ls /praktikum_Muhammad_Anwar_Ismail/hdfs/input
Found 1 items
-rw-r--r--   2 hadoop supergroup        132 2026-09-24 13:33 /praktikum_Muhammad_Anwar_Ismail/hdfs/input/input.txt

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug namenode
    Learn more at https://docs.docker.com/go/debug-cli/

D:\Semester 5\Big Data\Minggu ke-7>
```

Gambar 3.1.18 - Verifikasi kesiapan file input.txt di dalam HDFS klaster Hadoop.png

Perintah hdfs dfs -ls menemukan 1 item, yaitu input.txt berukuran 132 bytes dengan replikasi 2 pada direktori praktikum di HDFS. Keberadaan file ini memastikan data sumber siap dibaca oleh Spark.

```
scala> val fileHDFS =
     | sc.textFile("hdfs://namenode:8020/praktikum_Muhammad_Anwar_Ismail/hdfs/input/input.txt")
fileHDFS: org.apache.spark.rdd.RDD[String] = hdfs://namenode:8020/praktikum_Muhammad_Anwar_Ismail/hdfs/input/in
put.txt MapPartitionsRDD[12] at textFile at <console>:24

scala> fileHDFS.count()
res10: Long = 3

scala> fileHDFS.collect()
res11: Array[String] = Array(Hadoop HDFS DataNode MapReduce Big Data, Hadoop stores data as blocks and replicas
, Hadoop MapReduce processes data stored in HDFS)

scala>
```

Gambar 3.1.19 - Pembacaan dataset HDFS ke dalam RDD menggunakan sc.textFile.png

Perintah textFile memakai alamat hdfs://namenode:8020 untuk membaca input.txt menjadi RDD bertipe String. Action count() menghasilkan nilai 3 dan collect() menampilkan ketiga baris teks yang berisi istilah Hadoop, HDFS, DataNode, MapReduce, dan Big Data.

```
scala> val wordsHDFS = fileHDFS.flatMap(line ⇒ line.split("\\s+"))
wordsHDFS: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[13] at flatMap at <console>:23

scala> val pairsHDFS = wordsHDFS.map(word ⇒ (word, 1))
pairsHDFS: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[14] at map at <console>:23

scala> val countsHDFS = pairsHDFS.reduceByKey((a, b) ⇒ a + b)
countsHDFS: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[15] at reduceByKey at <console>:23

scala> countsHDFS.collect()
res12: Array[(String, Int)] = Array((blocks,1), (Big,1), (stores,1), (MapReduce,2), (data,2), (as,1), (DataNode
,1), (Data,1), (processes,1), (in,1), (HDFS,2), (and,1), (stored,1), (replicas,1), (Hadoop,3))

scala>
```

Gambar 3.1.20 - Eksekusi pipeline WordCount terdistribusi terhadap file HDFS.png

Pipeline flatMap dengan pemisah \s+, map, dan reduceByKey diterapkan pada RDD dari HDFS. Hasilnya memuat 15 kata unik, dengan Hadoop muncul 3 kali serta MapReduce, data, dan HDFS masing-masing 2 kali.

```
scala> val dataCache = fileHDFS.filter(line => line.nonEmpty).cache()
dataCache: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[16] at filter at <console>:23

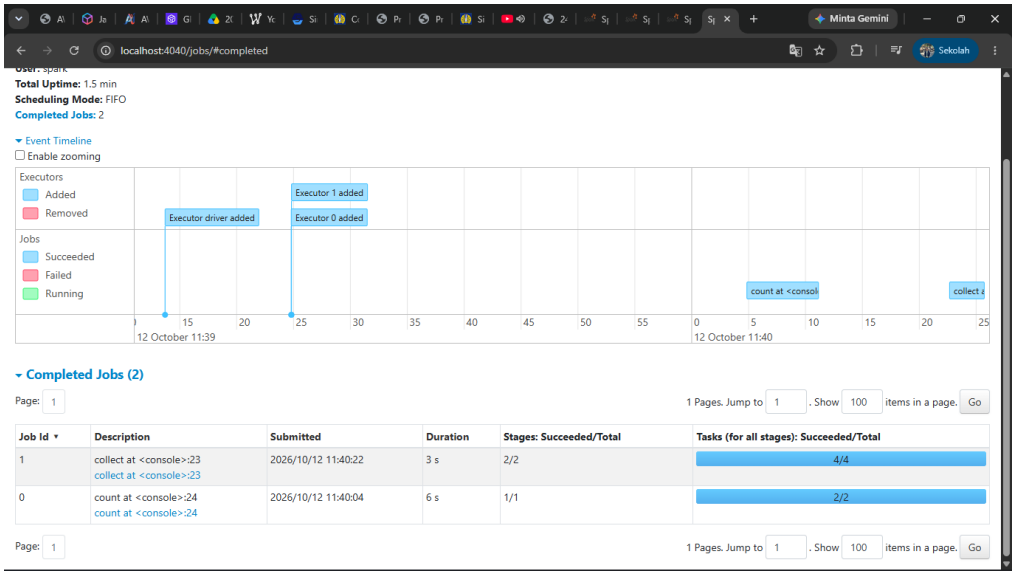
scala> dataCache.count()
res13: Long = 3

scala> dataCache.count()
res14: Long = 3

scala>
```

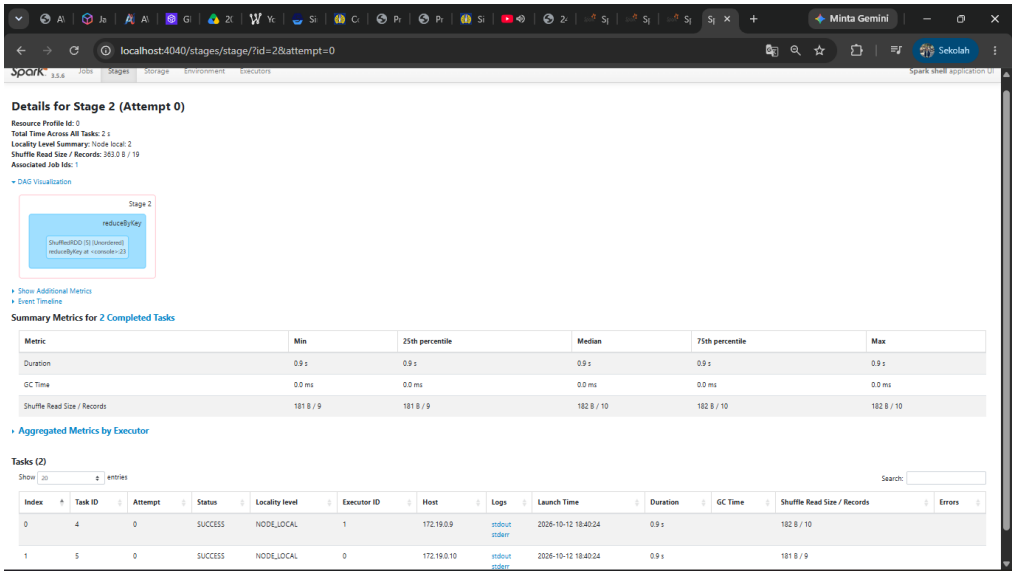
Gambar 3.1.21 - Pengujian mekanisme caching RDD menggunakan cache dan eksekusi dua aksi.png

RDD dataCache dibuat dari hasil filter baris tidak kosong dan ditandai dengan cache(). Dua kali pemanggilan count() menghasilkan nilai yang sama, yaitu 3, dan eksekusi kedua dapat memanfaatkan data yang sudah dipertahankan.



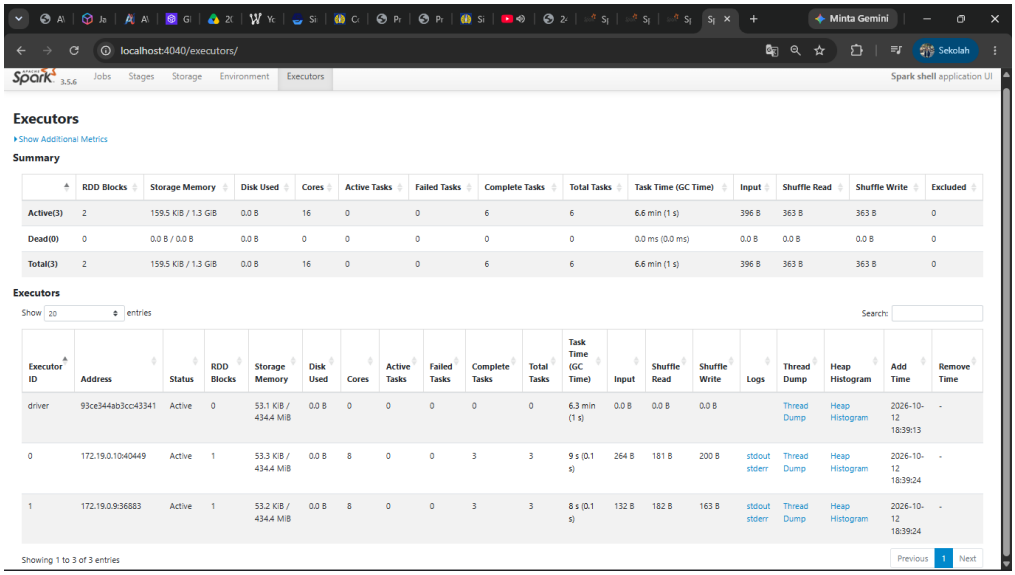
Gambar 3.1.22 - Antarmuka Spark Application UI pada localhost 4040 tab Jobs.png

Tab Jobs mencatat 2 completed jobs, yaitu count dengan 2 task selama 6 detik dan collect dengan 2 stage serta 4 task selama 3 detik. Event Timeline memperlihatkan Executor 0 dan Executor 1 ditambahkan saat aplikasi mulai berjalan.



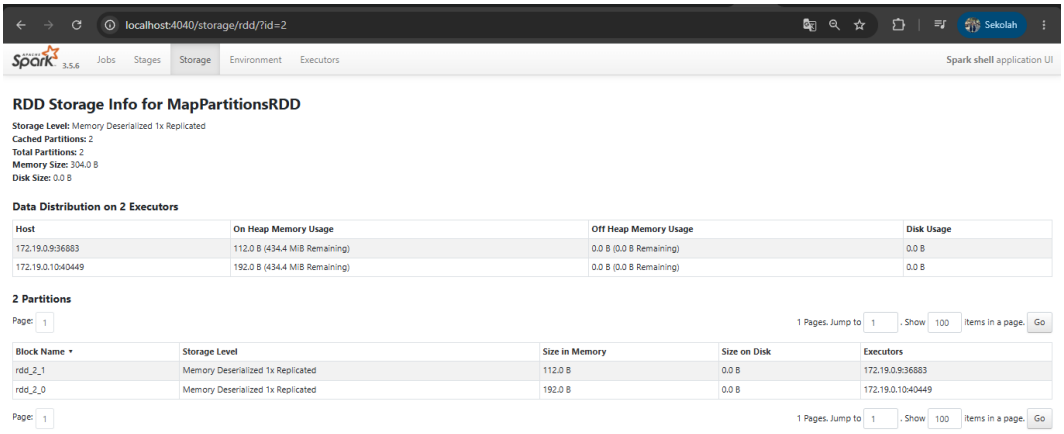
Gambar 3.1.23 - Antarmuka Spark Application UI tab Stages dan visualisasi DAG.png

Detail Stage 2 menampilkan DAG dengan satu operasi reduceByKey yang dijalankan oleh 2 task berstatus SUCCESS. Shuffle Read tercatat 363 B untuk 19 record, dan seluruh task berjalan pada level lokalitas NODE_LOCAL.



Gambar 3.1.24 - Antarmuka Spark Application UI tab Executors.png

Tab Executors menampilkan satu driver dan dua executor aktif dengan total 16 core. Sebanyak 6 task telah selesai tanpa kegagalan, dengan shuffle read dan shuffle write masing-masing 363 B.



Gambar 3.1.25 - Antarmuka Spark Application UI tab Storage saat RDD di-cache.png

Tab Storage menampilkan RDD yang di-cache dengan storage level Memory Deserialized 1x Replicated. Seluruh 2 dari 2 partisi tersimpan di memori sebesar 304 B dan tersebar pada kedua executor.

```
scala> :quit
spark@93ce344ab3cc:/opt/spark/work-dir$ exit
exit

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug spark-ma
ster
    Learn more at https://docs.docker.com/go/debug-cli/

D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>
```

Gambar 3.1.26 - Proses penghentian sesi spark-shell dan shutdown klaster Docker.png

Perintah :quit menutup sesi spark-shell, lalu perintah exit keluar dari container spark-master. Terminal kembali ke direktori spark-praktikum sehingga klaster siap dihentikan dengan docker compose down.

3.2 Hasil dan Tangkapan Layar Instalasi dan Menjalankan Apache Spark Standalone di Windows

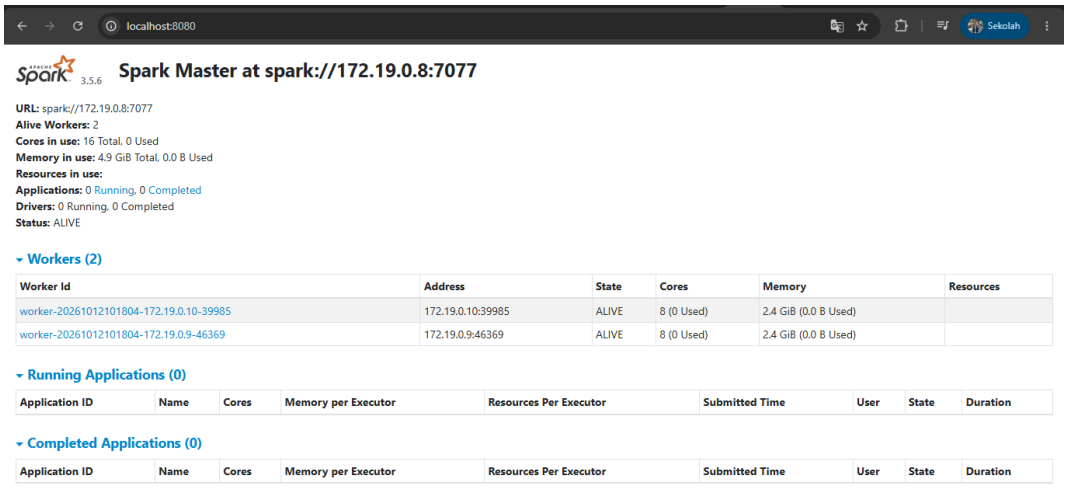
Bagian ini mendokumentasikan Praktikum 4, yaitu menjalankan klaster dengan 1 Master dan 2 Worker menggunakan Docker Compose, lalu memeriksa kondisinya pada Spark Master UI. Spark Shell kemudian dihubungkan ke Master untuk membuat RDD, menentukan jumlah partisi, menerapkan map dan filter, serta menjalankan WordCount. Hasil akhirnya mencakup pembacaan file HDFS dan tampilan Spark Application UI pada port 4040.

```
D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>docker compose -f docker-compose.spark.yml up -d
[+] up 3/3
 ✔Container spark-master   Running                                                    0.0s
 ✔Container spark-worker-1 Running                                                    0.0s
 ✔Container spark-worker-2 Running                                                    0.0s

D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>
```

Gambar 3.2.1 - Status container klaster Spark Standalone 1 Master dan 2 Worker aktif.png

Perintah docker compose -f docker-compose.spark.yml up -d memulai tiga container, yaitu spark-master, spark-worker-1, dan spark-worker-2. Ketiganya berstatus Running sehingga klaster 1 Master dan 2 Worker berhasil terbentuk.



Gambar 3.2.2 - Antarmuka Spark Master Web UI pada localhost 8080 dengan dua Worker berstatus ALIVE.png

Spark Master UI memperlihatkan dua Worker dengan state ALIVE, masing-masing berisi 8 core dan memori 2,4 GiB. Gambar ini menjadi bukti utama bahwa klaster memiliki dua Worker.

```
D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>docker exec -it spark-master bash
spark@93ce344ab3cc:/opt/spark/work-dir$ /opt/spark/bin/spark-shell --master spark://spark-master:7077
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/10/12 13:01:36 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builti
n-java classes where applicable
Spark context Web UI available at http://93ce344ab3cc:4040
Spark context available as 'sc' (master = spark://spark-master:7077, app id = app-20261012130140-0001).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.6
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 11.0.27)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

Gambar 3.2.3 - Menjalankan spark-shell terhubung ke Master Standalone.png

Spark Shell dijalankan dari dalam container spark-master dengan master spark://spark-master:7077 dan memperoleh application ID app-20261012130140-0001. Spark 3.5.6 dengan Scala 2.12.18 siap dipakai, dan peringatan NativeCodeLoader hanya menginformasikan bahwa library native Hadoop tidak dimuat.

```
scala> val data = Seq(1, 2, 3, 4, 5)
data: Seq[Int] = List(1, 2, 3, 4, 5)

scala> val rdd = sc.parallelize(data)
rdd: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[0] at parallelize at <console>:24

scala> rdd.collect()
res0: Array[Int] = Array(1, 2, 3, 4, 5)

scala>
```

Gambar 3.2.4 - Pembuatan RDD data numerik dengan parallelize dan penayangan data via collect.png

Koleksi Seq(1, 2, 3, 4, 5) diubah menjadi RDD melalui sc.parallelize. Action collect() menampilkan isi RDD berupa Array(1, 2, 3, 4, 5).

```
scala> rdd.getNumPartitions
res1: Int = 16

scala> val rdd4 = sc.parallelize(Seq(1, 2, 3, 4, 5, 6, 7, 8), 4)
rdd4: org.apache.spark.rdd.RDD[Int] = ParallelCollectionRDD[1] at parallelize at <console>:23

scala> rdd4.getNumPartitions
res2: Int = 4

scala> rdd4.glom().collect()
res3: Array[Array[Int]] = Array(Array(1, 2), Array(3, 4), Array(5, 6), Array(7, 8))

scala>
```

Gambar 3.2.5 - Penentuan jumlah partition RDD dan inspeksi pembagian data via glom.png

RDD pertama memiliki 16 partisi secara default, sedangkan RDD baru berisi angka 1 sampai 8 dibuat dengan 4 partisi. Perintah glom().collect() menunjukkan pembagian [1,2], [3,4], [5,6], dan [7,8].

```
scala> val hasil = rdd.map(x ⇒ x * 2)
hasil: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[3] at map at <console>:23

scala> hasil.collect()
res4: Array[Int] = Array(2, 4, 6, 8, 10)

scala> val genap = rdd4.filter(x ⇒ x % 2 == 0)
genap: org.apache.spark.rdd.RDD[Int] = MapPartitionsRDD[4] at filter at <console>:23

scala> genap.collect()
res5: Array[Int] = Array(2, 4, 6, 8)

scala>
```

Gambar 3.2.6 - Penerapan operasi transformasi map perkalian dua dan filter bilangan genap.png

Transformation map(x => x * 2) mengubah elemen 1 sampai 5 menjadi 2, 4, 6, 8, 10. Transformation filter pada RDD 4 partisi menghasilkan bilangan genap 2, 4, 6, 8.

```
scala> val teks = sc.parallelize(Seq("spark hadoop", "spark big data", "hadoop data"))
teks: org.apache.spark.rdd.RDD[String] = ParallelCollectionRDD[5] at parallelize at <console>:23

scala> val words = teks.flatMap(line ⇒ line.split(" "))
words: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[6] at flatMap at <console>:23

scala> val pairs = words.map(word ⇒ (word, 1))
pairs: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[7] at map at <console>:23

scala> val counts = pairs.reduceByKey((a, b) ⇒ a + b)
counts: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[8] at reduceByKey at <console>:23

scala> counts.collect()
res6: Array[(String, Int)] = Array((big,1), (hadoop,2), (data,2), (spark,2))

scala>
```

Gambar 3.2.7 - Eksekusi program WordCount sederhana menggunakan reduceByKey.png

Program WordCount memecah tiga kalimat, memasangkan setiap kata dengan angka 1, lalu menjumlahkannya melalui reduceByKey. Hasilnya adalah (big,1), (hadoop,2), (data,2), dan (spark,2) yang sesuai dengan hitungan manual.

```
scala> val file = sc.textFile("hdfs://namenode:8020/praktikum_Muhammad_Anwar_Ismail/hdfs/input/input.txt")
file: org.apache.spark.rdd.RDD[String] = hdfs://namenode:8020/praktikum_Muhammad_Anwar_Ismail/hdfs/input/input.
txt MapPartitionsRDD[10] at textFile at <console>:23

scala> file.count()
res7: Long = 3

scala> file.collect()
res8: Array[String] = Array(Hadoop HDFS DataNode MapReduce Big Data, Hadoop stores data as blocks and replicas,
 Hadoop MapReduce processes data stored in HDFS)

scala>
```

Gambar 3.2.8 - Pembacaan berkas input.txt dari HDFS menggunakan textFile.png

Berkas input.txt dibaca dari HDFS melalui alamat hdfs://namenode:8020 menjadi RDD String. Action count() menghasilkan 3 baris dan collect() menampilkan isi ketiga baris tersebut.

```
scala> val wordsHDFS = file.flatMap(line ⇒ line.split("\\s+"))
wordsHDFS: org.apache.spark.rdd.RDD[String] = MapPartitionsRDD[11] at flatMap at <console>:23

scala> val pairsHDFS = wordsHDFS.map(word ⇒ (word, 1))
pairsHDFS: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[12] at map at <console>:23

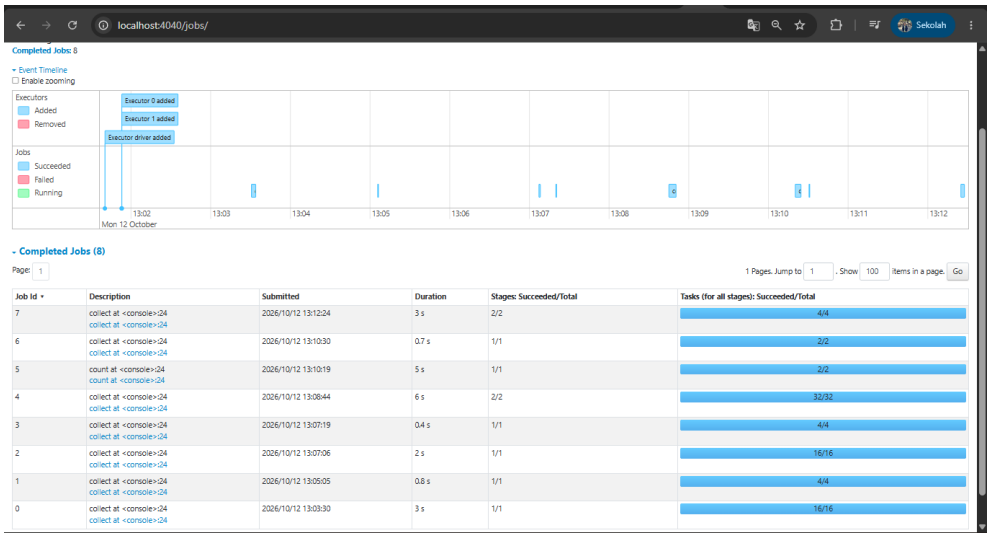
scala> val countsHDFS = pairsHDFS.reduceByKey((a, b) ⇒ a + b)
countsHDFS: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[13] at reduceByKey at <console>:23

scala> countsHDFS.collect()
res9: Array[(String, Int)] = Array((blocks,1), (Big,1), (stores,1), (MapReduce,2), (data,2), (as,1), (DataNode,
1), (Data,1), (processes,1), (in,1), (HDFS,2), (and,1), (stored,1), (replicas,1), (Hadoop,3))

scala>
```

Gambar 3.2.9 - Eksekusi WordCount terhadap berkas dari HDFS.png

Pipeline flatMap, map, dan reduceByKey dijalankan terhadap RDD hasil pembacaan HDFS. Keluarannya berisi 15 kata unik dengan Hadoop sebagai kata terbanyak sebanyak 3 kemunculan.



Gambar 3.2.10 - Antarmuka Spark Application UI pada port 4040.png

Tab Jobs mencatat 8 completed jobs yang seluruhnya berstatus sukses, dengan Executor 0 dan Executor 1 tampil pada Event Timeline. Jumlah task per job bervariasi, misalnya 16, 4, dan 2, mengikuti jumlah partisi RDD yang diproses.

```
scala> :quit
spark@93ce344ab3cc:/opt/spark/work-dir$ exit
exit

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug spark-ma
ster
    Learn more at https://docs.docker.com/go/debug-cli/

D:\Semester 5\Big Data\Minggu ke-7\spark-praktikum>
```

Gambar 3.2.11 - Penghentian sesi spark-shell.png

Perintah :quit menutup sesi spark-shell dan perintah exit keluar dari container spark-master. Terminal kembali ke direktori spark-praktikum sebagai tanda sesi selesai dengan bersih.

3.3 Analisis Mekanisme Partisi dan Distribusi Data (glom)

Perintah getNumPartitions menunjukkan jumlah partisi sebuah RDD, sedangkan glom() menampilkan isi setiap partisi sebagai array tersendiri. Tanpa penentuan jumlah partisi, RDD dari parallelize memperoleh 16 partisi, angka yang sama dengan total core klaster pada Spark Master UI. Pada tab Jobs, jumlah task per job mengikuti jumlah partisi, misalnya 16 task untuk RDD default dan 4 task untuk RDD berpartisi eksplisit, sehingga satu partisi diproses oleh satu task.

RDD	Jumlah Partisi	Isi Setiap Partisi (glom)
parallelize(1 to 20) tanpa parameter	16 (default, sama dengan total core)	Tidak ditampilkan
parallelize(1 to 20, 4)	4	[1-5], [6-10], [11-15], [16-20]
parallelize(Seq(1,2,...,8), 4)	4	[1,2], [3,4], [5,6], [7,8]

Data terbagi berurutan dan merata sehingga setiap task mendapat beban kerja yang seimbang. Pembagian ini juga memperlihatkan bahwa jumlah partisi dapat diatur sesuai kebutuhan dan tidak bergantung pada jumlah DataNode maupun jumlah executor.

3.4 Perbandingan Kinerja dan Karakteristik (map() vs flatMap())

Kedua operasi sama-sama berupa transformation yang menghasilkan MapPartitionsRDD dan dieksekusi pada masing-masing partisi tanpa perpindahan data antar node. Kinerja keduanya setara karena tidak memicu shuffle, sehingga perbedaan utama terletak pada jumlah elemen yang dihasilkan. Hasil praktikum menunjukkan map mempertahankan jumlah elemen, sedangkan flatMap mengubah dua kalimat menjadi lima kata.

Aspek	map()	flatMap()
Jenis operasi	Transformation (lazy)	Transformation (lazy)
Jumlah output per input	Tepat satu	Nol, satu, atau banyak
Struktur hasil	Satu elemen untuk setiap input	Hasil diratakan menjadi satu RDD
Tipe RDD hasil	MapPartitionsRDD	MapPartitionsRDD
Contoh pada praktikum	map(x => x * 5) pada 20 elemen menghasilkan 20 elemen	flatMap(split) pada 2 kalimat menghasilkan 5 kata
Peran dalam WordCount	Memasangkan kata dengan angka 1	Memecah baris menjadi kata

3.5 Perbedaan Analitis (reduce() vs reduceByKey())

Operasi reduce() adalah action yang menggabungkan seluruh elemen RDD dan langsung mengembalikan satu nilai ke driver, yaitu 210 untuk angka 1 sampai 20. Operasi reduceByKey() adalah transformation pada pasangan (key, value) yang menghasilkan RDD baru berisi satu nilai untuk setiap key. Tab Stages menunjukkan reduceByKey menghasilkan shuffle read sebesar 363 B untuk 19 record, karena data dengan key sama harus dikumpulkan terlebih dahulu.

Aspek	reduce()	reduceByKey()
Jenis operasi	Action	Transformation
Data masukan	Elemen tunggal, misalnya angka	Pasangan (key, value)
Hasil	Satu nilai	RDD berisi satu nilai per key
Tipe hasil pada praktikum	Int = 210	ShuffledRDD
Shuffle	Tidak tercatat pada tab Stages	Tercatat, yaitu shuffle read 363 B
Contoh pada praktikum	Total angka 1 sampai 20	(hadoop,2), (data,2), (spark,2)

3.6 Pembuktian Empiris Lazy Evaluation dan Efektivitas Caching

Pada Gambar 3.1.12 sampai 3.1.14, pembuatan RDD hasil map, filter, dan flatMap langsung menampilkan tipe RDD tanpa menampilkan data. Data baru muncul setelah action collect() atau count() dipanggil, dan setiap action tercatat sebagai satu job pada tab Jobs. Perilaku tersebut membuktikan bahwa Spark menunda eksekusi sampai hasil benar-benar diminta.

Tahap	Perintah	Respons Spark
Transformation	rddKali = rdd.map(x => x * 5)	Membuat MapPartitionsRDD tanpa menampilkan data
Transformation	rddGenap = rdd.filter(x => x % 2 == 0)	Membuat MapPartitionsRDD tanpa menampilkan data
Action	rddKali.collect()	Job dijalankan dan seluruh data tampil
Action	fileHDFS.count()	Job dijalankan dan hasil 3 tampil
Cache	dataCache = fileHDFS.filter(...).cache()	RDD ditandai untuk dipertahankan
Action berulang	dataCache.count() dua kali	Hasil sama, yaitu 3, dan tab Storage menampilkan RDD yang tersimpan

Efektivitas caching terlihat pada tab Storage, yaitu 2 dari 2 partisi tersimpan di memori sebesar 304 B dengan level Memory Deserialized 1x Replicated. Data yang sudah tersimpan dapat dipakai ulang oleh action berikutnya tanpa membaca dan menghitung ulang dari HDFS. Manfaat ini semakin besar pada RDD berukuran besar yang dipakai berkali-kali.

3.7 Hubungan HDFS Storage Layer dengan Spark Processing Layer

HDFS menyimpan input.txt berukuran 132 bytes dengan replikasi 2 pada DataNode, sedangkan Spark membacanya melalui textFile() dengan alamat hdfs://namenode:8020. Spark mengubah file tersebut menjadi RDD, membaginya ke dalam partisi, lalu menjalankan task pada executor untuk menghasilkan WordCount. Data mentah tetap berada di HDFS, sementara data yang sedang diproses berada pada resource executor.

Aspek	HDFS (Storage Layer)	Spark (Processing Layer)

Fungsi utama	Menyimpan data	Memproses data
Unit data	Block pada DataNode	Partition pada RDD
Pengelola	NameNode dan DataNode	Master, Worker, dan Executor
Bukti pada praktikum	input.txt tersimpan di HDFS dengan replikasi 2	RDD dari textFile menghasilkan 3 baris dan WordCount 15 kata unik
Antarmuka pemantauan	Port 9870 dan 8020	Port 8080 dan 4040

Kedua lapisan bekerja bergantian dalam satu alur, yaitu HDFS menyediakan data dan Spark mengolahnya secara paralel. Kombinasi ini memungkinkan data besar tersimpan aman dan tetap dapat dianalisis dengan cepat.

BAB IV
KESIMPULAN

4.1 Kesimpulan

Kesimpulan Praktikum ini berhasil menyiapkan dan menjalankan klaster Apache Spark 3.5.6 Standalone di atas Docker Windows dengan satu Master dan dua Worker. Seluruh node berjalan normal dengan status ALIVE yang terpantau langsung melalui Spark Master Web UI pada port 8080 serta Worker UI pada port 8081 dan 8082. Melalui sesi interaktif spark-shell, rangkaian operasi RDD berhasil dieksekusi, mulai dari pembuatan dataset lokal lewat parallelize(), transformasi data menggunakan map(), filter(), flatMap(), dan reduceByKey(), hingga penarikan hasil akhir dengan aksi collect(), count(), first(), serta reduce(). Fungsi getNumPartitions dan glom() memperlihatkan secara jelas bagaimana Spark membagi koleksi data ke dalam empat partisi terpisah agar dapat dikerjakan bersamaan oleh para executor.

Pengujian pada antarmuka Spark Application UI di port 4040 membuktikan cara kerja Lazy Evaluation, di mana baris transformasi yang ditulis hanya membentuk alur eksekusi di balik layar dan baru diproses nyata saat ada aksi yang dipanggil. Manfaat fitur cache() juga terlihat langsung pada tab Storage, dengan data partisi tersimpan di memori sebesar 304 byte sehingga pemanggilan aksi berikutnya berlangsung lebih singkat. Pada tahap integrasi, Spark berhasil membaca berkas input.txt langsung dari NameNode HDFS pada alamat hdfs://namenode:8020 untuk menjalankan perhitungan WordCount. Pengalaman mengatasi kendala pemetaan port 4040 pada docker-compose.spark.yml dan penyelarasan jaringan Docker mapreduce_default memberikan gambaran nyata tentang pentingnya konfigurasi jaringan antarkontainer dalam membangun arsitektur komputasi big data.

LAMPIRAN A
COMMAND UTAMA

No	Command	Fungsi
1	docker --version dan docker compose version	Memeriksa versi Docker Engine dan Docker Compose
2	docker compose -f docker-compose.spark.yml up -d	Menjalankan klaster Spark di latar belakang
3	docker ps	Menampilkan container yang sedang aktif
4	docker logs spark-master	Menampilkan log container untuk pelacakan masalah
5	docker exec -it spark-master bash	Masuk ke terminal container Spark Master
6	/opt/spark/bin/spark-submit --version	Memeriksa versi Apache Spark
7	/opt/spark/bin/spark-shell --master spark://spark-master:7077	Menjalankan Spark Shell yang terhubung ke Master
8	val rdd = sc.parallelize(data)	Membuat RDD dari koleksi data lokal
9	rdd.collect()	Menampilkan seluruh isi RDD ke driver
10	rdd.getNumPartitions	Menampilkan jumlah partisi RDD
11	rdd4.glom().collect()	Menampilkan isi setiap partisi RDD
12	rdd.map(x => x * 5)	Mengubah setiap elemen RDD menjadi nilai baru
13	rdd.filter(x => x % 2 == 0)	Menyaring elemen yang memenuhi kondisi
14	kalimat.flatMap(line => line.split(" "))	Memecah kalimat menjadi kata lalu meratakan hasilnya
15	rdd.reduce((a, b) => a + b)	Menjumlahkan seluruh elemen RDD menjadi satu nilai
16	pairs.reduceByKey((a, b) => a + b)	Menjumlahkan value berdasarkan key yang sama
17	sc.textFile("hdfs://namenode:8020/praktikum_Muhammad_Anwar_Ismail/hdfs/input/input.txt")	Membaca berkas teks dari HDFS menjadi RDD

18	hdfs dfs -ls /praktikum_Muhammad_Anwar_Ismail/hdfs/input	Memeriksa keberadaan file input pada HDFS
19	fileHDFS.filter(line => line.nonEmpty).cache()	Mempertahankan RDD untuk dipakai oleh beberapa action
20	docker compose -f docker-compose.spark.yml down	Menghentikan dan menghapus container klaster Spark

LAMPIRAN B
PORT YANG DIGUNAKAN

Layanan	Port Host dan Container	Fungsi
Spark Master (RPC)	7077:7077	Jalur koneksi Worker dan spark-shell ke Master
Spark Master UI	8080:8080	Memantau Master URL, Worker, CPU, dan memori klaster
Spark Worker 1 UI	8081:8081	Memantau kondisi Worker pertama
Spark Worker 2 UI	8082:8081	Memantau kondisi Worker kedua
Spark Application UI	4040:4040	Memantau Jobs, Stages, Storage, dan Executors saat spark-shell aktif